Climate & Urban Environment Statistics API

API Documentation

---

1. API Overview

The Climate & Urban Environment Statistics API is a RESTful web service designed to store and analyse historical environmental data for cities.

The system stores city information and environmental observations, including temperature and PM2.5 air pollution measurements. In addition to standard CRUD operations for city records, the API provides analytical endpoints for exploring environmental conditions over time.

These analytical features include:

- summary statistics for temperature and PM2.5
- trend analysis with moving averages
- anomaly detection using z-score, rolling z-score, and IQR methods
- environmental risk scoring
- regime classification based on rolling volatility
- comparison of environmental conditions between cities
- text-based environmental insights
- city rankings based on temperature, PM2.5, or risk score

The API is implemented using **FastAPI**, with **SQLAlchemy** for database interaction and **Pydantic** for request validation and response modelling. Responses are returned in JSON format.

---

2. Base URL

All requests should be made relative to the API base URL.

Local development:

http://127.0.0.1:8000

If deployed externally, the base URL will correspond to the deployed application endpoint.

Interactive API documentation is available at:

http://127.0.0.1:8000/docs

---

3. Data Models

The API is built around two main entities: **City** and **Observation**.

City

Represents a city stored in the database.

Field	Type	Description
id	integer	Unique identifier for the city
name	string	Name of the city
country	string	Country where the city is located
lat	float	Latitude coordinate
lon	float	Longitude coordinate

Each city can have multiple associated environmental observations.

A uniqueness constraint is applied to the combination of **name** and **country**, which prevents duplicate city records from being created.

Observation

Represents a recorded environmental observation for a city on a given date.

Field	Type	Description
id	integer	Observation identifier
city_id	integer	ID of the associated city
date	date	Date of the observation
temp_c	float or null	Temperature in degrees Celsius
pm25	float or null	PM2.5 air pollution concentration

In the database, the date field is stored as obs_date, but the API returns it as date.

A uniqueness constraint is applied to the combination of **city_id** and **obs_date**, ensuring that each city has at most one observation per date.

If a city is deleted, its associated observations are also deleted automatically through cascading delete behaviour.

---

4. Error Handling

The API uses standard HTTP status codes to indicate success or failure.

Status Code	Meaning
200	Request successful

Status Code	Meaning
201	Resource created successfully
204	Resource deleted successfully
404	Resource not found
409	Conflict, such as duplicate city creation
422	Invalid request parameters
500	Internal server error

The API uses two error response styles.

4.1 Problem Details style responses

Some custom errors, such as missing cities, are returned using a Problem Details style structure.

Example:

```
{
  "type": "https://example.com/problems/city-not-found",
  "title": "City not found",
  "status": 404,
  "detail": "City with id=999 does not exist.",
  "instance": "/cities/999"
}
```

4.2 FastAPI default validation errors

FastAPI's default validation error format is still used for framework-level request validation errors, such as missing required parameters, invalid data types, or values that fail built-in query constraints.

Example:

```
{
"detail": [
{
"type": "missing",
"loc": ["query", "start"],
"msg": "Field required",
"input": null
}
]
}
```

This means custom application errors use the Problem Details structure, while automatic request validation errors continue to use FastAPI's default validation format.

---

5. Endpoint Overview

The API exposes endpoints for city management, observation retrieval, and environmental analytics.

Method	Endpoint	Description
GET	/	Root endpoint showing API status
POST	/cities	Create a new city
GET	/cities	Retrieve all cities
GET	/cities/compare	Compare two cities over a selected date range
GET	/cities/{city_id}	Retrieve details for a specific city
PUT	/cities/{city_id}	Update an existing city
DELETE	/cities/{city_id}	Delete a city
POST	/observations	Create a new environmental observation for a city
GET	/observations	Retrieve observations with filtering, sorting, and pagination
GET	/cities/{city_id}/summary	Return summary statistics for a city
GET	/cities/{city_id}/trend	Return trend data and moving averages
GET	/cities/{city_id}/anomalies	Detect anomalies in a selected metric
GET	/cities/{city_id}/risk-score	Compute an environmental risk score
GET	/cities/{city_id}/regimes	Classify observations into stability regimes
GET	/cities/{city_id}/insights	Generate a text-based environmental insight
GET	/cities/rankings	Rank cities by temperature, PM2.5, or risk

Detailed parameter definitions and full response schemas are shown in the Swagger/OpenAPI documentation included in the appendix.

---

6. Example Endpoint

GET /cities/{city_id}/summary

Returns summary statistics for a selected city within a given date range.

Parameters

Parameter	Type	Description
city_id	integer	Unique city identifier

Parameter	Type	Description
start	date	Start date in YYYY-MM-DD format
end	date	End date in YYYY-MM-DD format

Example request

GET /cities/133/summary?start=2024-05-16&end=2024-05-21

Example response


```
{
  "city_id": 133,
  "start": "2024-05-16",
  "end": "2024-05-21",
  "count_days": 6,
  "avg_temp_c": 18.166666666666668,
  "min_temp_c": 14.0,
  "max_temp_c": 22.0,
  "avg_pm25": 6.3999999999999995,
  "min_pm25": 2.5,
  "max_pm25": 16.9
}
```


This endpoint aggregates temperature and PM2.5 values across the selected date range and returns summary statistics for the city.

---

7. Auto-Generated API Schema

The API uses FastAPI's built-in **OpenAPI / Swagger** documentation system.

This automatically generated documentation provides:

- complete endpoint listings
- query parameter definitions
- request body schemas
- response models
- validation rules
- example responses

While the API server is running, the Swagger interface can be accessed at:

http://127.0.0.1:8000/docs

COMP3011 Climate & Urban Environment Statistics API

1.0.0 OAS 3.1

/openapi.json

Stores historical city observations (temperature + PM2.5) and provides analytics endpoints.

Meta

GET / API status

Returns a simple message confirming that the API is running.

Parameters Try it out

No parameters

Responses

Code	Description	Links
200	Successful Response Media type application/json Controls `Accept` header. **Example Value** Schema `"string"`	*No links*

Cities

GET /cities List all cities

Returns all cities stored in the database, ordered by country and city name.

Parameters Try it out

No parameters

Responses

Code	Description	Links
200	Successful Response Media type application/json Controls `Accept` header. **Example Value** Schema	*No links*

```
[
  {
    "id": 0,
    "name": "string",
    "country": "string",
    "lat": 0,
    "lon": 0
  }
]
```

POST `/cities` Create a city

Creates a new city record with name, country, latitude, and longitude.

Parameters Try it out

No parameters

Request body required application/json

Example Value Schema

```
{
  "name": "string",
  "country": "string",
  "lat": -90,
```

```
  "lon": -180
}
```

Responses

Code	Description	Links
201	Successful Response Media type **application/json** Controls `Accept` header. **Example Value** Schema	*No links*
422	Validation Error Media type **application/json** **Example Value** Schema	*No links*

```
{
  "id": 0,
  "name": "string",
  "country": "string",
  "lat": 0,
  "lon": 0
}
```

```
{
  "detail": [
    {
      "loc": [
        "string",
        0
      ],
      "msg": "string",
      "type": "string",
      "input": "string",
      "ctx": {}
    }
  ]
}
```

GET `/cities/{city_id}` Get a city

Returns details for a single city by ID.

Parameters Try it out

Name	Description
city_id * required integer *(path)*	city_id

Responses

Code	Description	Links
200	Successful Response Media type **application/json** Controls `Accept` header. **Example Value** Schema	*No links*
422	Validation Error Media type **application/json** **Example Value** Schema	*No links*

```
{
  "id": 0,
  "name": "string",
  "country": "string",
  "lat": 0,
  "lon": 0
}
```

```
{
  "detail": [
    {
      "loc": [
        "string",
        0
      ],
      "msg": "string",
      "type": "string",
      "input": "string",
      "ctx": {}
    }
  ]
}
```

PUT `/cities/{city_id}` Update a city

Updates an existing city record.

Parameters

Try it out

Name	Description
city_id * required integer *(path)*	city_id

Request body required

application/json

Example Value Schema

```
{
  "name": "string",
  "country": "string",
  "lat": -90,
  "lon": -180
}
```

Responses

Code	Description	Links
200	Successful Response Media type application/json Controls `Accept` header. **Example Value** Schema `{ "id": 0, "name": "string", "country": "string", "lat": 0, "lon": 0 }`	*No links*
422	Validation Error Media type application/json **Example Value** Schema `{ "detail": [ { "loc": [`	*No links*

Code	Description	Links


```
            "string",
            0
          ],
          "msg": "string",
          "type": "string",
          "input": "string",
          "ctx": {}
        }
      ]
    }
```


DELETE `/cities/{city_id}` Delete a city

Deletes a city and its associated observations.

Parameters Try it out

Name	Description
city_id * required `integer` *(path)*	city_id

Responses

Code	Description	Links
204	Successful Response	*No links*
422	Validation Error	*No links*

Media type

application/json

Example Value Schema


```
{
  "detail": [
    {
      "loc": [
        "string",
        0
      ],
      "msg": "string",
      "type": "string",
      "input": "string",
      "ctx": {}
    }
```

Code	Description	Links
	] }	

Analytics

GET `/cities/compare` Compare two cities

Compares two cities over a selected date range using average temperature and PM2.5 statistics.

Parameters Try it out

Name	Description
city1 * required `integer` *(query)*	city1 *minimum: 1*
city2 * required `integer` *(query)*	city2 *minimum: 1*
start * required `string($date)` *(query)*	start
end * required `string($date)` *(query)*	end

Responses

Code	Description	Links

200	Successful Response
	Media type
	application/json
	Controls `Accept` header.
	Example Value Schema
	{
	"city_1": {
		No links

Code	Description	Links


```
    "city_id": 0,
    "city_name": "string",
    "country": "string",
    "start": "2026-03-13",
    "end": "2026-03-13",
    "count_days": 0,
    "avg_temp_c": 0,
    "avg_pm25": 0,
    "max_temp_c": 0,
    "max_pm25": 0
  },
  "city_2": {
    "city_id": 0,
    "city_name": "string",
    "country": "string",
    "start": "2026-03-13",
    "end": "2026-03-13",
    "count_days": 0,
    "avg_temp_c": 0,
    "avg_pm25": 0,
    "max_temp_c": 0,
    "max_pm25": 0
  },
  "hotter_city": "string",
  "more_polluted_city": "string",
  "temp_diff_avg": 0
```


422	Validation Error	*No links*

Media type

application/json

Example Value Schema


```
{
  "detail": [
    {
      "loc": [
        "string",
        0
      ],
      "msg": "string",
      "type": "string",
      "input": "string",
      "ctx": {}
    }
  ]
}
```


GET `/cities/rankings` Rank cities by metric

Ranks cities by average temperature, average PM2.5, or computed environmental risk over a selected date range.

Parameters Try it out

Name	Description
metric * required `string` *(query)*	metric *pattern: ^(temp_c|pm25|risk)$*
start * required `string($date)` *(query)*	start
end * required `string($date)` *(query)*	end
limit `integer` *(query)*	*Default value* : 10 10 *maximum: 50* *minimum: 1*

Responses

Code	Description	Links
200	Successful Response Media type **application/json** Controls `Accept` header. **Example Value** Schema	*No links*

```
{
  "metric": "string",
  "start": "2026-03-13",
  "end": "2026-03-13",
  "limit": 0,
  "rankings": [
    {
      "city_id": 0,
      "city_name": "string",
      "country": "string",
      "value": 0
    }
  ]
}
```

422	Validation Error Media type **application/json** **Example Value** Schema	*No links*

Code	Description	Links


```
{
  "detail": [
    {
      "loc": [
        "string",
        0
      ],
      "msg": "string",
      "type": "string",
      "input": "string",
      "ctx": {}
    }
  ]
}
```


GET `/cities/{city_id}/summary` Compute summary statistics

Returns average, minimum, and maximum temperature and PM2.5 values for a city within a selected date range.

Parameters Try it out

Name	Description
city_id * required `integer` *(path)*	city_id
start * required `string($date)` *(query)*	start
end * required `string($date)` *(query)*	end

Responses

Code	Description	Links
200	Successful Response Media type **application/json** Controls `Accept` header.	*No links*

Code	Description	Links

Example Value Schema


```
{
  "city_id": 0,
  "start": "2026-03-13",
  "end": "2026-03-13",
  "count_days": 0,
  "avg_temp_c": 0,
  "min_temp_c": 0,
  "max_temp_c": 0,
  "avg_pm25": 0,
  "min_pm25": 0,
  "max_pm25": 0
}
```


422 Validation Error *No links*

Media type

application/json

Example Value Schema


```
{
  "detail": [
    {
      "loc": [
        "string",
        0
      ],
      "msg": "string",
      "type": "string",
      "input": "string",
      "ctx": {}
    }
  ]
}
```


GET `/cities/{city_id}/trend` Get trend data

Returns time-series values and moving averages for temperature or PM2.5 over a selected date range.

Parameters Try it out

Name	Description
city_id * required `integer` *(path)*	city_id
metric `string` *(query)*	*Default value* : temp_c temp_c *pattern: ^(temp_c|pm25)$*

Name	Description
start * required `string($date)` *(query)*	start
end * required `string($date)` *(query)*	end
window `integer` *(query)*	*Default value* : 7 7 *maximum: 60* *minimum: 1*

Responses

Code	Description	Links
200	Successful Response Media type **application/json** Controls `Accept` header. **Example Value** Schema	*No links*
422	Validation Error Media type **application/json** **Example Value** Schema	*No links*

```
{
  "city_id": 0,
  "metric": "string",
  "start": "2026-03-13",
  "end": "2026-03-13",
  "window": 0,
  "points": [
    {
      "date": "2026-03-13",
      "value": 0,
      "moving_avg": 0
    }
  ]
}
```

```
{
  "detail": [
    {
      "loc": [
        "string",
```

Code	Description	Links


```
            0
          ],
          "msg": "string",
          "type": "string",
          "input": "string",
          "ctx": {}
        }
      ]
    }
```


GET `/cities/{city_id}/anomalies` Detect anomalies

Detects unusual temperature or PM2.5 values using z-score, rolling z-score, or IQR methods.

Parameters Try it out

Name	Description
city_id * required `integer` *(path)*	city_id
metric `string` *(query)*	*Default value* : temp_c temp_c *pattern: ^(temp_c|pm25)$*
start * required `string($date)` *(query)*	start
end * required `string($date)` *(query)*	end
method `string` *(query)*	*Default value* : zscore zscore *pattern: ^(zscore|rolling_zscore|iqr)$*
threshold `number` *(query)*	*Default value* : 2 2 *maximum: 10* *minimum: 0.5*
window `integer` *(query)*	*Default value* : 14 14 *maximum: 120* *minimum: 3*

Name	Description
k number *(query)*	*Default value* : 1.5 1.5 *maximum: 10* *minimum: 0.5*

Responses

Code	Description	Links
200	Successful Response Media type **application/json** Controls `Accept` header. **Example Value** Schema	*No links*

```
{
  "city_id": 0,
  "metric": "string",
  "start": "2026-03-13",
  "end": "2026-03-13",
  "method": "zscore",
  "window": 0,
  "threshold": 0,
  "k": 0,
  "mean": 0,
  "std": 0,
  "q1": 0,
  "q3": 0,
  "iqr": 0,
  "anomalies": [
    {
      "date": "2026-03-13",
      "value": 0,
      "score": 0,
      "direction": "high"
    }
  ]
}
```

422	Validation Error Media type **application/json** **Example Value** Schema	*No links*

```
{
  "detail": [
    {
      "loc": [
        "string",
        0
      ],
```

Code	Description	Links


```
        "msg": "string",
        "type": "string",
        "input": "string",
        "ctx": {}
      }
    ]
  }
```


GET `/cities/{city_id}/risk-score` Compute risk score

Computes a simple environmental risk score using variability, anomaly rate, and trend slope.

Parameters Try it out

Name	Description
city_id * required `integer` *(path)*	city_id
start * required `string($date)` *(query)*	start
end * required `string($date)` *(query)*	end

Responses

Code	Description	Links
200	Successful Response Media type **application/json** Controls `Accept` header. **Example Value** Schema	*No links*


```
{
  "city_id": 0,
  "start": "2026-03-13",
  "end": "2026-03-13",
  "risk_score": 0,
  "label": "Low",
```

Code	Description	Links


```
  "components": {
    "additionalProp1": 0,
    "additionalProp2": 0,
    "additionalProp3": 0
  }
}
```


422 Validation Error *No links*

Media type

application/json

Example Value Schema


```
{
  "detail": [
    {
      "loc": [
        "string",
        0
      ],
      "msg": "string",
      "type": "string",
      "input": "string",
      "ctx": {}
    }
  ]
}
```


GET `/cities/{city_id}/regimes` Classify environmental regimes

Classifies observations as Stable, Volatile, or Extreme using rolling standard deviation.

Parameters Try it out

Name	Description
city_id * required `integer` *(path)*	city_id
metric `string` *(query)*	*Default value* : temp_c temp_c *pattern: ^(temp_c|pm25)$*
start * required `string($date)` *(query)*	start
end * required `string($date)` *(query)*	end

Name	Description
window integer *(query)*	*Default value* : 14 14 *maximum: 120* *minimum: 3*

Responses

Code	Description	Links
200	Successful Response Media type **application/json** Controls `Accept` header. **Example Value** Schema	*No links*


```
{
  "city_id": 0,
  "metric": "string",
  "start": "2026-03-13",
  "end": "2026-03-13",
  "window": 0,
  "points": [
    {
      "date": "2026-03-13",
      "value": 0,
      "rolling_std": 0,
      "regime": "Stable"
    }
  ]
}
```

Code	Description	Links
422	Validation Error	*No links*

Media type

application/json

Example Value Schema


```
{
  "detail": [
    {
      "loc": [
        "string",
        0
      ],
      "msg": "string",
      "type": "string",
      "input": "string",
      "ctx": {}
    }
  ]
}
```


GET `/cities/{city_id}/insights` Generate environmental insight

Returns a short natural-language summary of environmental conditions and overall risk level.

Parameters Try it out

Name	Description
city_id * required `integer` *(path)*	city_id
start * required `string($date)` *(query)*	start
end * required `string($date)` *(query)*	end

Responses

Code	Description	Links
200	Successful Response Media type **application/json** Controls `Accept` header. **Example Value** Schema	*No links*

```
{
  "city_id": 0,
  "city_name": "string",
  "start": "2026-03-13",
  "end": "2026-03-13",
  "insight": "string"
}
```

422	Validation Error Media type **application/json** **Example Value** Schema	*No links*

```
{
  "detail": [
    {
      "loc": [
        "string",
        0
      ],
      "msg": "string",
      "type": "string",
      "input": "string",
      "ctx": {}
    }
  ]
}
```

Observations

GET `/observations` Query observations

Returns environmental observations for a city with optional date filtering, sorting, and pagination.

Parameters Try it out

Name	Description
city_id * required `integer` *(query)*	city_id *minimum: 1*

Name	Description
start string | (string | null)($date) *(query)*	start
end string | (string | null)($date) *(query)*	end
limit integer *(query)*	*Default value* : 200 200 *maximum: 2000* *minimum: 1*
offset integer *(query)*	*Default value* : 0 0 *minimum: 0*
sort string *(query)*	*Default value* : obs_date obs_date *pattern: ^(obs_date|temp_c|pm25)$*
order string *(query)*	*Default value* : asc asc *pattern: ^(asc|desc)$*

Responses

Code	Description	Links
200	Successful Response Media type **application/json** Controls `Accept` header. **Example Value** Schema	*No links*

```
[
  {
    "id": 0,
    "city_id": 0,
    "obs_date": "2026-03-13",
    "temp_c": 0,
    "pm25": 0
  }
]
```

Code	Description	Links
422	Validation Error	*No links*

Media type

application/json

Example Value Schema


```
{
  "detail": [
    {
      "loc": [
        "string",
        0
      ],
      "msg": "string",
      "type": "string",
      "input": "string",
      "ctx": {}
    }
  ]
}
```


POST `/observations` Create an observation

Creates a new environmental observation for a city on a specific date.

Parameters Try it out

No parameters

Request body required **application/json**

Example Value Schema


```
{
  "city_id": 0,
  "date": "2026-03-13",
  "temp_c": 0,
  "pm25": 0
}
```


Responses

Code	Description	Links
201	Successful Response	*No links*

Code	Description	Links

Media type

application/json

Controls `Accept` header.

Example Value Schema

```
"string"
```

422 Validation Error *No links*

Media type

application/json

Example Value Schema

```
{
  "detail": [
    {
      "loc": [
        "string",
        0
      ],
      "msg": "string",
      "type": "string",
      "input": "string",
      "ctx": {}
    }
  ]
}
```

Schemas

AnomalyOut > Expand all `object`

AnomalyPoint > Expand all `object`

CityCompareOut > Expand all `object`

CityCompareStats > Expand all `object`

CityCreate > Expand all `object`

CityInsightsOut > Expand all `object`

CityOut > Expand all `object`

CityRankingItem > Expand all `object`

CityRankingOut > Expand all `object`

CitySummaryOut > Expand all `object`

CityUpdate > Expand all `object`

HTTPValidationError > Expand all `object`

ObservationCreateIn > Expand all `object`

ObservationOut > Expand all `object`

RegimePoint > Expand all `object`

RegimesOut > Expand all `object`

RiskScoreOut > Expand all `object`

TrendOut > Expand all `object`

TrendPoint > Expand all `object`

ValidationError > Expand all `object`